RAMANUJAN PRIMES BEYOND 10^{19}: A CERTIFIED EXTENSION OF OEIS A181671 VIA A 128-BIT SEGMENTED SIEVE AND BRACKETED ANALYTIC TAIL BOUNDS

GAURAV VYAS

Abstract. A Ramanujan prime is the smallest integer R_n such that $\pi(x) - \pi(x/2) \geq n$ for all $x \geq R_n$, where π is the ordinary prime-counting function. The count of Ramanujan primes below 10^k, tabulated as OEIS sequence A181671, has published values only through $k = 17$; two further terms ($k = 18, 19$) exist as unpublished prior computations. We present a certified extension of this sequence to $k = 20, 21, 22, 23$, computed on consumer hardware with no institutional resources. The method combines a bracketing lemma that reduces the certification of $f(x) = \pi(x) - \pi(x/2)$ on an entire interval to exact evaluations of π at finitely many endpoints, a custom 128-bit segmented sieve for the region immediately above 10^k (since general-purpose sieve libraries are limited to 2^{64}), and the explicit analytic bounds of Dusart [3] and Johnston [4] to certify the tail beyond the sieved and bracketed region without further computation. Every new term's defining anchors $\pi(Q)$ and $\pi(Q/2)$ are cross-verified by two independent algorithms (Gourdon and Deleglise–Rivat) implemented in the `primecount` library [5], and selected results are further verified against an independent published reference for $\pi(x)$ and by direct Miller–Rabin primality testing that bypasses the custom sieve entirely. We report the full implementation, the RAM- and CPU-aware scheduling required to complete these computations reliably on hardware with 16–32 GB of RAM, and an empirical time-complexity analysis showing the method scales as ~ 3.5–$3.7\times$ per decade of Q once measurement artifacts from interrupted runs are removed from the data — better than the naïve $Q^{2/3}$ asymptotic predicted for the underlying prime-counting algorithm.

1. Introduction

Ramanujan's 1919 paper [1] proved a strengthening of Bertrand's postulate: for every $n \geq 1$ there exists a least integer R_n such that

$$\pi(x) - \pi(x/2) \geq n \qquad \text{for all } x \geq R_n.$$

These R_n are now called Ramanujan primes. The number of Ramanujan primes not exceeding 10^k, denoted $a(k)$, is tabulated in the OEIS as sequence A181671 [2]. As of this writing the OEIS entry publishes verified values only for $k \leq 17$; the values $a(18)$ and $a(19)$ circulate as unpublished, self-computed extensions with no independent verification on record.

The obstacle to extending this sequence further is entirely computational, not mathematical: the natural method is to sieve the interval $[10^k, 10^k + W]$ for some window W large enough to certify that the running count of $f(x) = \pi(x) - \pi(x/2)$ never dips below $a(k) - 1$ again. Under the naïve approach W grows with $\sqrt{Q}$, and general-purpose prime-sieving libraries such as `primesieve` [6] are limited to 64-bit integers, so this approach becomes infeasible well before $Q = 10^{20}$: at that scale the certification window alone spans on the order of 10^{12} integers, entirely above the 2^{64} boundary.

This paper reports a certified extension of $a(k)$ to $k = 20, 21, 22, 23$. Every term is accompanied by a machine-checkable certificate and passes multiple layers of independent verification described in Section 5. The entire computation was carried out on two personal machines (Section 3.5) with no cluster, cloud, or institutional compute access.

Date: August 14, 2026.

Email: `gaurav.vyas.1729@gmail.com`.

2. Background

2.1. **Ramanujan primes and the bracketing identity.** Write $f(x) = \pi(x) - \pi(x/2)$. Note that f is *not* monotone — it increases by 1 at each prime and decreases by 1 at each doubled prime, and its local dips below $f(Q)$ are precisely what the certification must rule out (or locate exactly). What *is* monotone is π itself, and that alone yields the two elementary facts on which the entire method rests.

Proposition 1 (Counting identity)**.** *For every integer $x \geq 2$, the number of Ramanujan primes $\leq x$ equals $\min_{y \geq x} f(y)$, and the minimum is attained.*

Proof. Since f is integer-valued and $f(y) \to \infty$, the minimum over $y \geq x$ is attained. By definition R_n is the least integer such that $f(y) \geq n$ for all $y \geq R_n$. If $R_n \leq x$ then $f(y) \geq n$ for all $y \geq x$; conversely, if $f(y) \geq n$ for all $y \geq x$ then $R_n \leq x$ by minimality of R_n. Hence $\{\, n : R_n \leq x \,\} = \{\, n : n \leq \min_{y \geq x} f(y) \,\}$, which is an initial segment of cardinality $\min_{y \geq x} f(y)$. □

Lemma 1 (Bracketing lemma)**.** *For $a \leq y \leq b$,*

$$f(y) \;\geq\; \pi(a) - \pi(b/2) \;=\; f(a) - \big[\pi(b/2) - \pi(a/2)\big].$$

Proof. π is nondecreasing, so $\pi(y) \geq \pi(a)$ and $\pi(y/2) \leq \pi(b/2)$ for $a \leq y \leq b$; subtract. □

This is elementary, but it is the entire mechanism that makes evaluation at $Q = 10^{20}$ and beyond tractable: a single exact evaluation of π at each of two endpoints lower-bounds f across the whole interval $[a, b]$, with no sieving of the interior required. Because f drifts upward at rate $\sim 1/(2 \log Q)$ in practice, the bound survives *geometric* growth of the interval width, so only $O(\log(W/D))$ grid points are needed to bracket a window of width W starting from an initial exactly-sieved region of width D.

2.2. **Analytic tail bounds.** Grid bracketing alone still requires evaluating π out to some finite Y before the interval can be declared safe. Beyond Y, we use two published unconditional (RH-independent) explicit bounds on π:

- **Johnston (2022)** [4]: using a computer-verified region of the Riemann hypothesis up to height 3×10^{12},

  $$|\pi(x) - \mathrm{li}(x)| < \frac{\sqrt{x}\,\log x}{8\pi} \qquad \text{for } 2657 \leq x \leq 1.101 \times 10^{26}.$$

  Applying this at both $x = y$ and $x = y/2$ (requiring $y \geq 5314$) gives the certifiable lower bound used in this work,

  $$G(y) \;=\; \mathrm{li}(y) - \mathrm{li}(y/2) - \frac{\sqrt{y}\log y + \sqrt{y/2}\log(y/2)}{8\pi} \;\leq\; f(y).$$

- **Dusart (2010)** [3], Theorem 6.9: for $x \geq 88{,}783$,

  $$\pi(x) \geq \frac{x}{\ln x}\left(1 + \frac{1}{\ln x} + \frac{2}{\ln^2 x}\right),$$

  and for $x \geq 2{,}953{,}652{,}287$,

  $$\pi(x) \leq \frac{x}{\ln x}\left(1 + \frac{1}{\ln x} + \frac{2.334}{\ln^2 x}\right).$$

  Writing $\mathrm{lb}(x)$ and $\mathrm{ub}(x)$ for these lower and upper bounds respectively, the far tail is controlled by $\Delta(y) := \mathrm{lb}(y) - \mathrm{ub}(y/2) \leq f(y)$, valid for $y \geq 1.101 \times 10^{26}$. These bounds are what ultimately limit this method's reach: since the Johnston bound requires $y \leq 1.101 \times 10^{26}$, the sequence becomes uncertifiable by this pipeline once Q approaches that limit — which occurs at $a(26)$ (Section 7).

Both bounds were transcribed into the implementation from their respective published papers and re-verified against the primary sources as part of this work (Section 5).

The pipeline evaluates G and Δ only at *single points*: it exits its grid loop at the first grid point Y with $\lfloor G(Y) \rfloor \geq m$, and it checks $\Delta(1.101 \times 10^{26}) \geq m$ once. For those pointwise checks to certify the entire tail, both functions must be nondecreasing beyond the point checked. This is true, and worth stating explicitly since the soundness of the certificate depends on it:

Lemma 2 (Tail monotonicity)**.** *G is strictly increasing on $[5314, 1.101 \times 10^{26}]$, and Δ is strictly increasing on $[1.101 \times 10^{26}, \infty)$.*

Proof. For G: with $L = \log y$,

$$G'(y) \;=\; \frac{1}{L} - \frac{1}{2\log(y/2)} - E'(y) \;=\; \frac{L - 2\log 2}{2L\log(y/2)} - E'(y) \;\geq\; \frac{L-2\log 2}{2L^2} - \frac{L+2}{8\pi\sqrt{y}},$$

where $E(y)$ is the error term of G and the bound $E'(y) \leq (L+2)/(8\pi\sqrt{y})$ follows from $\frac{d}{dy}\sqrt{y}\log y = (\log y + 2)/(2\sqrt{y})$ applied to both summands of E. Thus $G'(y) > 0$ whenever $4\pi\sqrt{y}\,(L - 2\log 2) > L^2(L+2)$, i.e. whenever $4\pi \cdot \frac{\sqrt{y}}{L^2} \cdot \frac{L-2\log 2}{L+2} > 1$. At $y = 5314$ the left side is ≈ 8.5, and both factors $\sqrt{y}/L^2$ (increasing for $y > e^4$) and $(L - 2\log 2)/(L+2)$ are increasing, so the inequality holds on the whole interval.

For Δ: with $L = \log y$ and $\ell = \log(y/2) = L - \log 2$, using $\frac{d}{dy}\,(y/L^k) = (L-k)/L^{k+1}$,

$$\Delta'(y) \;\geq\; \frac{L-1}{L^2} \;-\; \frac{1}{2\ell}\Big(1 + \frac{1}{\ell} + \frac{2.334}{\ell^2}\Big),$$

where the first term keeps only the leading summand of lb' (the others are positive for $L > 3$) and the second over-estimates ub' term by term (using $1 + 1/\ell + 2.334/\ell^2 \leq 1 + 2/\ell$ for $\ell \geq 3$). Cross-multiplying, and using $L - 1 \leq \ell \leq L$, positivity follows from $2(L-1)^3 > L^2(L+2)$, which holds for all $L \geq 8$; here $L \geq \log(1.101 \times 10^{26}) \approx 59.96$. □

The full certification argument now assembles as follows.

Theorem 1 (Certificate soundness)**.** *Fix integers $Q \geq 5314$ and $D \geq 1$, and suppose the pipeline terminates with output m, final grid point $Y \leq 1.101 \times 10^{26}$, and $\Delta(1.101 \times 10^{26}) \geq m$. Then*

$$\#\{\textit{Ramanujan primes} \leq Q\} \;=\; \min_{y \geq Q} f(y) \;=\; m.$$

Proof. By construction m is the exact minimum of f on $[Q, Q+D]$ (the sieved walk evaluates every change of f there, including the starting value $f(Q)$), so $\min_{y \geq Q} f(y) \leq m$ and it suffices to show $f(y) \geq m$ for $y > Q + D$. On each grid interval $[y_i, y_{i+1}]$ the pipeline verifies $f(y_i) - [\pi(y_{i+1}/2) - \pi(y_i/2)] \geq m$, which bounds f on the whole interval by Lemma 1; these intervals tile $[Q + D, Y]$. For $y \in [Y, 1.101 \times 10^{26}]$, $f(y) \geq G(y) \geq G(Y) \geq m$ by Lemma 2 and the loop's exit condition. For $y \geq 1.101 \times 10^{26}$, $f(y) \geq \Delta(y) \geq \Delta(1.101 \times 10^{26}) \geq m$, again by Lemma 2. Combining with Proposition 1 gives the claim. □

3. Implementation

3.1. **Why a custom 128-bit sieve was necessary.** The exact-count region $[Q, Q + D]$ immediately above Q must be sieved directly rather than bracketed, since the bracketing lemma alone gives no information about where inside an interval f might dip. At $Q = 10^{20}$ this region already lies entirely above $2^{64} \approx 1.8 \times 10^{19}$, outside the range of `primesieve` and every off-the-shelf sieve library available at the time of writing. We wrote a custom 128-bit segmented sieve, `sieve128.cpp`, implementing:

- a standard segmented sieve of Eratosthenes over 128-bit unsigned integers, using `primesieve` internally only to generate the base primes up to $\sqrt{Q + D}$ (which remain well within 64-bit range even at $Q = 10^{23}$, since $\sqrt{10^{23}} \approx 3.16 \times 10^{11}$);

- an exact walk of f across the segmented region, tracking the running minimum and the offset at which it occurs, so that the certificate records not just the final count but the exact point (if any) where f dips within the sieved window;
- cross-validation against `primesieve` below 2^{64} (exact agreement on prime lists, including across the 2^{64} boundary) and against `sympy`'s primality routines above 2^{64}, both performed as part of this project's own validation suite before any new term was attempted.

3.2. **Software architecture.** The pipeline consists of three components:

(1) `sieve128.cpp/.exe` — the 128-bit segmented sieve and exact walk, described above.
(2) `primecount` (upstream, unmodified except for build configuration) — provides exact, arbitrary-precision $\pi(x)$ via two independently implemented algorithms, Gourdon's method and the Deleglise–Rivat method, used here purely as a cross-check against each other rather than for speed.
(3) `ramanujan128.py` — the orchestration driver: computes and caches every π value obtained, invokes the sieve walk near each new Q, evaluates the analytic tail bounds, and assembles a JSON certificate per term recording every anchor, checkpoint, and verification outcome.

3.3. **Crash-safe persistent caching.** Because a single anchor evaluation at $Q = 10^{23}$ can run for several hours (Section 6), and because this project's compute budget consisted of ordinary desktop/laptop machines subject to reboots, power interruptions, and resource contention with unrelated workloads, every exact π value ever computed is persisted to `pi_cache.json` using an atomic write pattern: results are written to a temporary file and then renamed over the target, so that a process kill or power loss mid-write can never leave the cache in a corrupted state. This was validated directly by simulating fifteen mid-write process kills, all of which left the cache file intact. In practice, this pipeline survived at least one full unattended power interruption and two accidental process terminations during the computation of the terms reported here, in every case resuming from exactly the last completed value with no lost work beyond whatever single call was in flight at the moment of interruption.

3.4. **RAM- and CPU-aware adaptive parallelism.** Measured memory footprint per concurrent `primecount` call grows substantially faster than the algorithm's own $\sqrt[3]{x}$ working-set prediction: on the hardware described in Section 3.5, peak RAM per call was measured directly at approximately 0.36 GB at $Q = 5 \times 10^{20}$, 1.26 GB at $Q = 10^{22}$, and 5.9 GB at $Q = 10^{23}$ — an empirical scaling exponent of roughly 0.4–0.7 in Q, not the 1/3 the underlying algorithm's complexity would suggest. Running multiple checkpoint evaluations concurrently is necessary to make full use of available CPU cores (a single `primecount` call was measured to use only $\sim$4 of 12 available logical cores, since its dominant phases are not fully parallel), but naively launching as many concurrent calls as there are cores risks exhausting system RAM and crashing the host — which was observed directly once during this project, when RAM contention with an unrelated concurrent workload coincided with a full system crash. The driver therefore queries live available RAM before sizing each batch of concurrent evaluations, computes an expected per-call footprint from the fitted empirical curve above with a safety margin, and caps concurrency accordingly, trading some throughput for reliability on hardware without dedicated headroom.

3.5. **Cross-machine reproduction and migration.** Terms $a(1)$ through $a(22)$ were computed on a laptop-class machine (13th Gen Intel Core i5-13420H, 8 cores/12 threads, 16 GB RAM, NVIDIA GeForce RTX 4050 Laptop GPU, not used for this computation). Term $a(23)$ was computed on a separate desktop machine (12th Gen Intel Core i5-12400F, 6 cores/12 threads, 32 GB RAM, NVIDIA GeForce RTX 3060, likewise not used for this computation), after the laptop's available RAM proved insufficient for $Q = 10^{23}$: a single `primecount` call at that scale was measured to require approximately 5.9 GB, leaving under 1 GB of headroom on the 13.7 GB effectively available on the laptop for the many hours the computation would require, and the attempt was deliberately

cancelled rather than risked. The persistent JSON cache and all source files transferred verbatim between machines (verified by SHA-256 hash equality before resuming), so no computation already completed was repeated.

As a byproduct, this migration produced a genuine independent cross-machine reproduction: $a(18)$ and $a(19)$ were recomputed from the persisted cache on the desktop machine using a from-scratch driver invocation, and matched the laptop-computed values exactly on every intermediate quantity (Table 1).

TABLE 1. Cross-machine independent reproduction of $a(18)$, $a(19)$.

Quantity	$k = 18$	$k = 19$
$\pi(10^k)$	24,739,954,287,740,860	234,057,667,276,344,607
$\pi(10^k/2)$	12,585,956,566,571,620	118,959,989,688,273,472
$a(k)$	12,153,997,721,169,239	115,097,677,588,071,134

4. RESULTS

Table 2: $a(k)$ for $k = 1, \ldots, 23$. Times for $k \leq 18$ and $k = 21, 22$ are clean, single-pass, cache-bypassed measurements; $k = 19, 20$ are genuine first-time measurements with no interruption. See Section 6 for why some originally recorded times were superseded by fresh recomputation. $k = 23$'s recorded time reflects only its final continuous run segment, not the full multi-day effort across earlier interruptions (same caveat as originally applied to $k = 21, 22$; see Section 6). Status codes are defined below the table.

k	$a(k)$	time	chk.	status
1	1	0.64 s	0	A
2	10	0.38 s	0	A
3	72	0.38 s	0	A
4	559	0.38 s	0	A
5	4,459	0.38 s	0	A
6	36,960	0.37 s	0	A
7	316,066	0.36 s	0	A
8	2,760,321	0.36 s	0	A
9	24,491,666	0.35 s	0	A
10	220,098,288	0.34 s	0	A
11	1,998,400,235	0.35 s	0	A
12	18,299,775,876	0.41 s	1	A
13	168,773,875,190	0.59 s	3	A
14	1,566,017,986,235	1.40 s	6	A
15	14,606,736,768,049	4.41 s	8	A
16	136,860,923,837,558	16.57 s	10	A
17	1,287,462,389,890,262	91.81 s	13	A
18	12,153,997,721,169,239	5 m 43 s	15	B
19	115,097,677,588,071,134	11 m 18 s	17	B
20	1,093,039,678,770,734,297	63 m 59 s	16	C
21	10,406,559,368,229,726,028	4 h 31 m 47 s	18	D
22	99,306,360,875,818,676,888	14 h 43 m 22 s	12	C
23	949,634,047,203,617,038,366	26 h 38 m 31 s†	14	F

Status codes: **A** = OEIS A181671 published value. **B** = prior self-computed, unpublished; cross-machine reproduced in this work (Table 1). **C** = new result; both anchors cross-algorithm verified (Gourdon vs. Deleglise–Rivat). **D** = new

result; cross-algorithm verified *and* independently reproduced via direct Miller–Rabin walk testing (Section 5). **F** = new result; cross-algorithm verified, independently reproduced via direct Miller–Rabin walk testing, *and* its primary anchor $\pi(Q)$ matches an independently published external reference value (Section 5) — the most thoroughly verified term in this table.

† Not a clean measurement: this run resumed from checkpoints cached during earlier interrupted attempts and ran concurrently with an unrelated workload on the same machine (Section 3.5), so 26 h 38 m understates the true from-scratch cost. Unlike $a(21)$ and $a(22)$, $a(23)$ has *not* been re-verified with a clean, cache-bypassed, single-pass run; do not use this figure in any time-complexity comparison (Section 6 already excludes it for this reason).

4.1. $a(23)$. $Q = 10^{23}$ was certified on the desktop machine described in Section 3.5. Its defining minimum occurs at offset 42 above Q (a dip of exactly 1 below $f(Q)$) — the second term in this work, after $a(21)$, whose minimum is not simply $\pi(Q) - \pi(Q/2)$. An independently published reference value for $\pi(10^{23})$, from the Wikipedia prime-counting-function table [7], matches the certified primary anchor exactly: $\pi(10^{23}) = 1{,}925{,}320{,}391{,}606{,}803{,}968{,}923$. The largest Ramanujan prime $\leq 10^{23}$, $99{,}999{,}999{,}999{,}999{,}999{,}999{,}947$, was independently confirmed prime via deterministic Miller–Rabin testing. The value $a(23)$ itself is certified with the same rigor as every other term in this table; only its *recorded wall-clock time* is unreliable (Table 2, note †) and excluded from the performance analysis in Section 6.

5. Verification

Every new term reported here ($k = 18$–23) passes the following independent checks, none of which are internal self-consistency checks against the pipeline's own prior output:

(1) **Cross-algorithm anchor verification.** $\pi(Q)$ and $\pi(Q/2)$ for $k = 20, 21, 22, 23$ (and, redundantly, for the cross-machine reproduction of $k = 18, 19$) were each computed by two independently implemented algorithms within `primecount` (Gourdon and Deleglise–Rivat); all agreed exactly.

(2) **Independent external reference.** The Wikipedia prime-counting-function table [7], an independent source predating and unrelated to this project, lists $\pi(10^{19})$ through $\pi(10^{23})$; all five match this work's computed values digit-for-digit.

(3) **Miller–Rabin walk reproduction.** $a(21)$'s defining minimum occurs at an offset of 58 above $Q = 10^{21}$, a dip of 3 below $f(Q)$; $a(23)$'s occurs at an offset of 42 above $Q = 10^{23}$, a dip of exactly 1. Both were reproduced independently of `sieve128` by direct primality testing (via `sympy`) of every integer in the relevant ranges: for $a(21)$, 0 primes above and exactly 3 below; for $a(23)$, 0 primes above and exactly 1 below — confirming both dips via entirely different machinery. $a(20)$ and $a(22)$ have their minimum at offset 0 (i.e. $a(k) = \pi(Q) - \pi(Q/2)$ exactly), so no walk verification is applicable to them.

(4) **Analytic agreement.** For $k = 20, 21, 22, 23$, $\mathrm{li}(Q) - \mathrm{li}(Q/2)$ agrees with the certified $f(Q)$ to within 10^{-10} relative difference in every case (respectively 8.2×10^{-11}, 2.1×10^{-11}, 5.3×10^{-12}, and 4.5×10^{-12}, computed at 60-digit working precision).

(5) **Growth-ratio consistency.** $a(k)/a(k-1)$ for $k = 19, \ldots, 23$ is 9.4699, 9.4966, 9.5208, 9.5427, and 9.5627 respectively — a smooth, monotonically increasing sequence consistent with the expected $\sim 9.5\times$ per-decade growth from prime density arguments, with no discontinuity at the transition into the cross-verified regime.

(6) **Analytic tail-bound formulas checked against source.** The Johnston (2022) and Dusart (2010) bounds as implemented were compared directly against the published statements in [4, 3]; both the constants and the stated domains of validity match exactly (Section 2).

(7) **Fresh, cache-bypassed single-pass recomputation.** $a(21)$ and $a(22)$ were originally computed across multiple interrupted sessions, so their first recorded wall-clock times reflected only a final resumed pass rather than a clean measurement. Both were independently

recomputed from scratch, bypassing the persistent cache entirely, and matched the original certified values exactly, additionally producing publication-quality clean timings (Table 2).

(8) **Record-prime primality.** The largest Ramanujan prime below each certified bound (e.g. 99,999,999,999,999,999,989 for $k = 20$) was independently confirmed prime via deterministic Miller–Rabin testing.

6. Performance analysis

An initial analysis performed during this project, using timings contaminated by interrupted and resumed runs under RAM pressure, suggested per-decade cost growth as high as 11–26×, well above the naïve $Q^{2/3} \approx 4.64\times$-per-decade asymptotic predicted for the underlying algorithm. This estimate was an artifact of the measurement conditions, not a property of the algorithm, and is superseded here. Using the clean, single-pass timings in Table 2 (which for $k = 18$–22 specifically bypass the persistent cache and any partial-run contamination), a log-linear regression of ln(time) against k gives:

- $k = 13$–19 (single-algorithm anchors): $\times 3.51$ per decade of Q;
- $k = 20$–22 (cross-verified anchors, roughly 3–4× slower per call than single-algorithm since Deleglise–Rivat is measurably slower than Gourdon at equal Q): $\times 3.72$ per decade of Q.

Both regimes land in the same 3.5–3.7×-per-decade range despite the methodology difference, corresponding to time $\propto Q^{0.55\text{–}0.57}$ — *better* than the $Q^{2/3}$ asymptotic usually quoted for this class of prime-counting algorithm. The $k = 23$ timing is deliberately excluded from both regressions: like the original (superseded) $k = 21, 22$ measurements, it reflects only the final continuous run segment of a computation spread across several interrupted sessions and two concurrent workloads, so it is not a clean single-pass data point (Table 2, caption). The number of grid-bracketing checkpoints required grows far more slowly, consistent with the $O(\log(W/D))$ prediction of Lemma 1, confirming that per-call cost of each `primecount` evaluation, not the count of evaluations needed, is what dominates the growth in total wall time.

7. Limitations and the method's ceiling

Two independent constraints bound how far this method can be pushed.

Mathematical ceiling. The Johnston (2022) bound used for the analytic tail is only valid for $y \le 1.101 \times 10^{26}$. Beyond that, this pipeline falls back to the weaker Dusart (2010) bounds, which remain valid but require checking `dusart_ok`(m) can certify $f(y) \ge m$ for all sufficiently large y — a condition that is expected to fail once Q approaches 1.101×10^{26}, placing the method's absolute mathematical ceiling at approximately $a(26)$. Beyond that, stronger published tail bounds would be required, not merely faster hardware.

Practical ceiling. Independent of the mathematical limit, per-call RAM at $Q = 10^{23}$ was measured at ∼5.9 GB and is increasing with an empirical exponent of roughly 0.4–0.7 in Q (Section 3), well above what fits comfortably alongside an operating system and other workloads on consumer hardware with 16–32 GB of RAM. This was the direct cause of the mid-project migration described in Section 3.5, and is expected to bind again well before the mathematical ceiling at $a(26)$ is reached, absent either a machine with substantially more RAM or algorithmic improvements to `primecount`'s own memory footprint.

8. Conclusion

We have presented a certified, multiply-verified extension of OEIS A181671 to $k = 20, 21, 22, 23$, computed entirely on consumer-grade personal hardware. The approach combines an elementary bracketing lemma with published, unconditional analytic tail bounds to avoid sieving computationally infeasible ranges, backed by a purpose-built 128-bit segmented sieve for the region that does require direct primality information. Beyond the mathematical result, this work required nontrivial

systems engineering — crash-safe persistent caching, RAM-aware adaptive parallelism, and verified cross-machine migration — to complete reliably outside an institutional computing environment, which we have documented here as it materially affects the reproducibility of the reported timings and the practical ceiling on how far the method can currently be pushed.

Acknowledgments

The author thanks the maintainers of `primecount` and `primesieve`, whose exact, high-performance implementations of $\pi(x)$ made this work possible.

AI Disclosure

This paper's prose was drafted with the assistance of an AI system (Claude, Anthropic), directed and reviewed throughout by the author, who verified every mathematical claim, transcribed formula, and reported result against primary sources and independent computation (Section 5). The implementation — `ramanujan128.py`, `sieve128.cpp`, and the surrounding build and orchestration tooling — was likewise developed with AI assistance (Claude, Anthropic) acting as a coding collaborator, with the author directing the design and independently testing and verifying the resulting code, including all cross-checks reported in this paper. The author takes full responsibility for the correctness of all results reported here.

References

[1] S. Ramanujan, *A proof of Bertrand's postulate*, Journal of the Indian Mathematical Society **11** (1919), 181–182.
[2] OEIS Foundation Inc., *A181671: Number of Ramanujan primes* $\leq 10^n$, The On-Line Encyclopedia of Integer Sequences, https://oeis.org/A181671.
[3] P. Dusart, *Estimates of some functions over primes without R.H.*, arXiv:1002.0442, 2010.
[4] D. R. Johnston, *Improving bounds on prime counting functions by partial verification of the Riemann hypothesis*, The Ramanujan Journal (2022); arXiv:2109.02249.
[5] K. Walisch et al., *primecount: fast prime counting function implementations*, https://github.com/kimwalisch/primecount.
[6] K. Walisch et al., *primesieve: fast prime number generator*, https://github.com/kimwalisch/primesieve.
[7] Wikipedia contributors, *Prime-counting function*, Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Prime-counting_function.

Email address: gaurav.vyas.1729@gmail.com